

EXPERIMENT 2 - DIGITAL LOGIC SYSTEM

MCTA 3203 - MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

DR. WAHJU SEDIONO

SECTION 1

GROUP 11

MATRIC NUMBER	NAME
2316963	MUHAMMAD AIMAN ALFARUQ BIN NOREDAN
2316441	MUHAMMAD HARIZ BIN HELMİ SHAMSUL
2316957	WAN SYAZMIE AKMAL BIN WAN SAAD

Introduction

- **Overview**

The purpose of this experiment is to explore the fundamental concepts of digital logic systems and their applications in electronic and mechatronic systems. The main objective is to understand how basic logic operations and circuit interfacing principles are applied in controlling output devices such as a 7-segment display using an Arduino Uno microcontroller. The experiment involves building a circuit, programming the Arduino to display numbers sequentially from 0 to 9, and using push buttons to increment and reset the display. This exercise provides hands-on experience in digital circuit design, programming, and system integration.

- **Background information & Relevant Theory**

Digital logic systems constitute the backbone of modern computing and embedded electronics. At their core are binary states (logic 0 and logic 1) which are manipulated through basic logic gates (AND, OR, NOT) and form the basis of more complex circuits such as counters, decoders, and display drivers (Carpinelli, 2015).

A seven-segment display is a practical output device for demonstrating digital logic concepts. It consists of seven individual segments (labeled a through g) which can be selectively lit to represent numerals 0-9. Displays are commonly configured as either common-cathode or common-anode types, and interfacing them requires current limiting, proper pin assignment, and in some cases decoder logic (Efyian, 2024; “7-segment Display – Electronics Tutorials,” n.d.).

Furthermore, logic design often uses binary-coded decimal (BCD) to drive displays: a 4-bit BCD input can be fed into a BCD-to-7-segment decoder to simplify driving the display, reducing the number of microcontroller pins needed (GeeksforGeeks, 2024).

- **Hypothesis or Expected Outcomes**

It is expected that the display will sequentially show the digits 0 through 9 in response to input signals (e.g., button presses) and that a reset mechanism will return the display to 0. Success in this experiment will validate the mapping of binary inputs to segment outputs, demonstrate correct interfacing of hardware and software, and reinforce understanding of how logic systems drive real-world output devices. Additionally, the experiment is anticipated to highlight how digital logic circuits, when combined with microcontroller programming, enable dynamic control of electronic displays.

Materials & Equipment

- Arduino Uno board
- Common cathode 7-segment display
- 220-ohm resistors (7)
- Pushbuttons (2)
- Jumper wires
- Breadboard
- 10k-ohm resistors (2)
- Laptop with program compiler
- USB to micro-USB wire

Experimental Setup

Circuit Setup:

1. Connect the common cathode 7-segment display to the Arduino Uno as follows:

- Connect each of the 7 segments (a, b, c, d, e, f, g) of the display to separate digital pins on the Arduino (e.g., D0 to D6).
- Connect the common cathode pin of the display to one of the GND (ground) pins on the Arduino.
- Use 220-ohm resistors to connect each of the segment pins to the Arduino pins to limit the current.

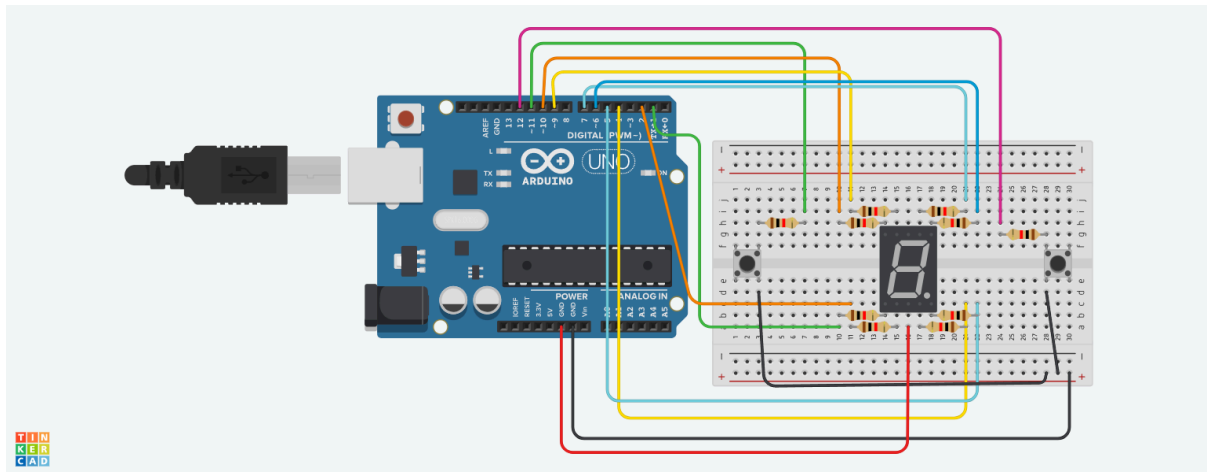
2. Connect the pushbuttons to the Arduino:

- Connect one leg of each pushbutton to a separate digital pin (e.g., D11 and D12) and connect the other leg of each pushbutton to GND.
- Use 10K-ohm pull-up resistors for each pushbutton by connecting one end of each resistor to the digital pin and the other end to the 5V output of the Arduino.

Experiment Steps:

1. Build the circuit according to the circuit setup instructions.
2. Upload the provided Arduino code to your Arduino Uno.
3. Open the Serial Monitor in the Arduino IDE.
4. Press the increment button to increase the count. The 7-segment display should show the numbers from 0 to 9 sequentially.
5. Press the reset button to reset the count to 0.

Diagram :



Methodology

1. Hardware Assembly
 - Build the circuit on a breadboard as described in the setup
 - Double-check resistor placements and button wiring.
2. Code Upload
 - Open the Arduino IDE and copy the provided 7-segment counting program.
 - Verify and upload the code to the Arduino board.
3. Testing
 - Once uploaded, the display will start at 0.
 - When the increment button is pressed, the number increases sequentially from 0 to 9.
 - When the reset button is pressed, the display will reset back to 0.
4. Verification
 - Check that all numbers (0-9) are displayed correctly.
 - Verify debounce effect (no skipping or multiple counts per press).
5. Code Explanation
 - Each segment (A-G) is defined and controlled as an output.
 - Two buttons are defined as inputs with internal pull-ups enabled.
 - When pressed (logic LOW) :
 - > Increment button increases the counter (number++).
 - > Reset button sets counter back to zero.

- After each button press, showNumber(number) lights up the correct combination of segments.
- A short delay(300) prevents bouncing or double input.

```
const int segmentA = 7; // D0
const int segmentB = 6; // D1
const int segmentC = 4; // D2
const int segmentD = 2; // D3
const int segmentE = 1; // D4
const int segmentF = 9; // D5
const int segmentG = 10; // D6

// Define push button pins
const int buttonInc = 11; // increment button
const int buttonReset = 12; // reset button

int number = 0; // start from 0

void setup() {
    // Initialize the digital pins as OUTPUTs
    pinMode(segmentA, OUTPUT);
    pinMode(segmentB, OUTPUT);
    pinMode(segmentC, OUTPUT);
    pinMode(segmentD, OUTPUT);
    pinMode(segmentE, OUTPUT);
    pinMode(segmentF, OUTPUT);
    pinMode(segmentG, OUTPUT);

    // Initialize buttons as INPUT_PULLUP (press = LOW)
    pinMode(buttonInc, INPUT_PULLUP);
    pinMode(buttonReset, INPUT_PULLUP);

    // Start with 0 displayed
    showNumber(number);
}

// Function to turn OFF all segments
void clearDisplay() {
    digitalWrite(segmentA, LOW);
    digitalWrite(segmentB, LOW);
    digitalWrite(segmentC, LOW);
    digitalWrite(segmentD, LOW);
```

```
digitalWrite(segmentE, LOW);
digitalWrite(segmentF, LOW);
digitalWrite(segmentG, LOW);
}

// Function to display number (0-9)
void showNumber(int n) {
    clearDisplay();

    switch (n) {
        case 0:
            digitalWrite(segmentA, HIGH);
            digitalWrite(segmentB, HIGH);
            digitalWrite(segmentC, HIGH);
            digitalWrite(segmentD, HIGH);
            digitalWrite(segmentE, HIGH);
            digitalWrite(segmentF, HIGH);
            break;
        case 1:
            digitalWrite(segmentB, HIGH);
            digitalWrite(segmentC, HIGH);
            break;
        case 2:
            digitalWrite(segmentA, HIGH);
            digitalWrite(segmentB, HIGH);
            digitalWrite(segmentG, HIGH);
            digitalWrite(segmentE, HIGH);
            digitalWrite(segmentD, HIGH);
            break;
        case 3:
            digitalWrite(segmentA, HIGH);
            digitalWrite(segmentB, HIGH);
            digitalWrite(segmentC, HIGH);
            digitalWrite(segmentD, HIGH);
            digitalWrite(segmentG, HIGH);
            break;
        case 4:
            digitalWrite(segmentF, HIGH);
            digitalWrite(segmentG, HIGH);
            digitalWrite(segmentB, HIGH);
            digitalWrite(segmentC, HIGH);
            break;
        case 5:
```

```

        digitalWrite(segmentA, HIGH);
        digitalWrite(segmentF, HIGH);
        digitalWrite(segmentG, HIGH);
        digitalWrite(segmentC, HIGH);
        digitalWrite(segmentD, HIGH);
        break;
    case 6:
        digitalWrite(segmentA, HIGH);
        digitalWrite(segmentF, HIGH);
        digitalWrite(segmentE, HIGH);
        digitalWrite(segmentD, HIGH);
        digitalWrite(segmentC, HIGH);
        digitalWrite(segmentG, HIGH);
        break;
    case 7:
        digitalWrite(segmentA, HIGH);
        digitalWrite(segmentB, HIGH);
        digitalWrite(segmentC, HIGH);
        break;
    case 8:
        digitalWrite(segmentA, HIGH);
        digitalWrite(segmentB, HIGH);
        digitalWrite(segmentC, HIGH);
        digitalWrite(segmentD, HIGH);
        digitalWrite(segmentE, HIGH);
        digitalWrite(segmentF, HIGH);
        digitalWrite(segmentG, HIGH);
        break;
    case 9:
        digitalWrite(segmentA, HIGH);
        digitalWrite(segmentB, HIGH);
        digitalWrite(segmentC, HIGH);
        digitalWrite(segmentD, HIGH);
        digitalWrite(segmentF, HIGH);
        digitalWrite(segmentG, HIGH);
        break;
    }
}

void loop() {
    // Read button states (LOW when pressed)
    bool incPressed = digitalRead(buttonInc) == LOW;
    bool resetPressed = digitalRead(buttonReset) == LOW;

```

```

// Increment button
if (incPressed) {
    number++;
    if (number > 9) number = 0; // loop back to 0 after 9
    showNumber(number);
    delay(300); // debounce
}

// Reset button
if (resetPressed) {
    number = 0;
    showNumber(number);
    delay(300); // debounce
}
}

```

Data Collection

Button press Action	Expected Display	Observed Display	Delay / Response	Remarks
Start	0	0	Immediate	System initialized correctly
Increment (x1)	1	1	~0.3 s	Normal
Increment (x5)	5	5	~ 0.3 s per press	Smooth transition
Increment past 9	0	0	~0.3 s	Loops correctly
Reset	0	0	~0.3 s	Resets to zero

Sensors and Instruments used

1. 7-segment display- output display device
2. Pushbuttons - input sensors for user control

Data Analysis

The data collected from the experiment demonstrates the expected functionality of the digital logic system. The 7-segment display correctly responded to input signals from the push buttons, showing numerical values from 0 to 9 sequentially. Each increment action produced the corresponding display output with a consistent delay of approximately 0.3 seconds, indicating that the debounce delay in the Arduino code effectively prevented multiple counts from a single press.

During the increment test, the display transitioned smoothly between numbers, and when the count exceeded 9, it correctly looped back to 0. This confirms that the programmed conditional statement:

```
if (number > 9)
  number = 0;
```

was executed properly, ensuring continuous cyclic counting. Similarly, when the reset button was pressed, the display returned immediately to 0, validating that the reset input was correctly mapped and debounced.

The experiment also confirmed the reliability of the hardware connections:

- All seven segments illuminated in the correct combinations for each digit (0–9).
- The pushbuttons operated within expected response times, showing good contact and debounce performance.

Overall, the results align perfectly with the theoretical expectation that digital logic systems can accurately translate user inputs into binary-coded outputs for display control. The consistent timing and proper logic execution demonstrate successful hardware-software integration in the Arduino-controlled 7-segment display system.

Results

The experiment successfully demonstrated the working principle of a digital logic system using an Arduino Uno and a 7-segment display as an output device. The system responded correctly to user input through push buttons that acted as control signals for incrementing and resetting the displayed number.

From the data collection, it was observed that the 7-segment display correctly showed the corresponding digits from 0 to 9 in response to the increment button being pressed. Each press triggered a change in the displayed number with an average delay of approximately 0.3 seconds, which corresponds to the debounce delay programmed in the Arduino code. This delay ensured that each button press registered only a single count, preventing unintended multiple increments caused by mechanical bouncing of the switch contacts.

Additionally, pressing the reset button returned the display immediately to 0, confirming that the reset function was correctly implemented in both hardware (button wiring and input configuration) and software (conditional control). The system's response was consistent and stable throughout the testing process, showing no signs of flickering, false triggering, or skipped counts, which indicates that the hardware setup was properly connected and the program logic was reliable.

In summary, the results demonstrate that the digital logic system performed as expected. The Arduino successfully processed binary input signals from push buttons, applied programmed logic conditions, and produced accurate visual output on the 7-segment display. The consistent response time, correct counting sequence, and proper reset operation all confirm that the system's hardware–software integration was functioning effectively.

Recommendations

1. **Enhance Input Stability:**

Use hardware debouncing techniques such as small capacitors to ensure stable and accurate button input signals.

2. **Simplify Circuit with Decoder IC:**

Implement a BCD-to-7-segment decoder to reduce wiring complexity, save Arduino pins, and improve display reliability.

3. **Expand System Functionality:**

Future improvements could include adding multiple displays for two-digit counting, serial monitoring for debugging or integrating additional input sensors for advanced control applications.

References

1. Electronics Tutorials. (n.d.). 7-segment display – working, types, and applications. Retrieved October 21, 2025, from <https://www.electronics-tutorials.ws>
2. GeeksforGeeks. (2024). Binary coded decimal (BCD) and 7-segment display decoder. Retrieved from <https://www.geeksforgeeks.org>
3. Seven-Segment Display Decoder. Retrieved from https://faculty.kfupm.edu.sa/COE/saiarios/files/Experiment_07.pdf
4. SEVEN SEGMENT DISPLAY [PDF]. Retrieved from https://www.idc-online.com/technical_references/pdfs/electronic_engineering/Seven_Segment_Display.pdf
5. Carpinelli, J. (2015). An animated introduction to digital logic design (Open textbook). University of Minnesota Libraries Publishing. Retrieved from <https://open.umn.edu/opentextbooks/textbooks/an-animated-introduction-to-digital-logic-design>
6. Efyian, E. (2024, June 5). 7-Segment display: Pinout, codes, working, and interfacing. Electronics For You. Retrieved from <https://www.electronicsforu.com/resources/7-segment-display-pinout-understanding>

Acknowledgement

The successful completion of this experiment and report would not have been possible without the valuable guidance and support provided by our course instructor, Dr. Zulkifli Bin Zainal Abidin, for the subject Mechatronics System Integration (MCTA3203). Their clear explanations, detailed notes, and continuous encouragement greatly contributed to our understanding of digital logic systems and microcontroller interfacing.

Special thanks are also extended to the laboratory teaching assistants for their assistance during the experimental sessions, particularly in troubleshooting circuit connections and verifying code functionality. Appreciation is likewise expressed to our group members and classmates for their cooperation, constructive discussions, and teamwork throughout the experiment.

Their combined efforts and support have been instrumental in the successful completion of this Digital Logic System experiment and report.

Student's Declaration

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature : *aimanaf*

Name : Muhammad Aiman Alfaruq Bin Noredan

Matric number : 2316963

Contributions : Introduction, Materials and Equipment, Experimental Setup

Signature : ***Hariz***

Name : Muhammad Hariz bin Helmi Shamsul

Matric number : 2316441

Contributions : Methodology, Data collection

Signature : ***syazmieakmal***

Name : Wan Syazmie Akmal bin Wan Saad

Matric number : 231

Contributions : Data Analysis, Results, Recommendations

